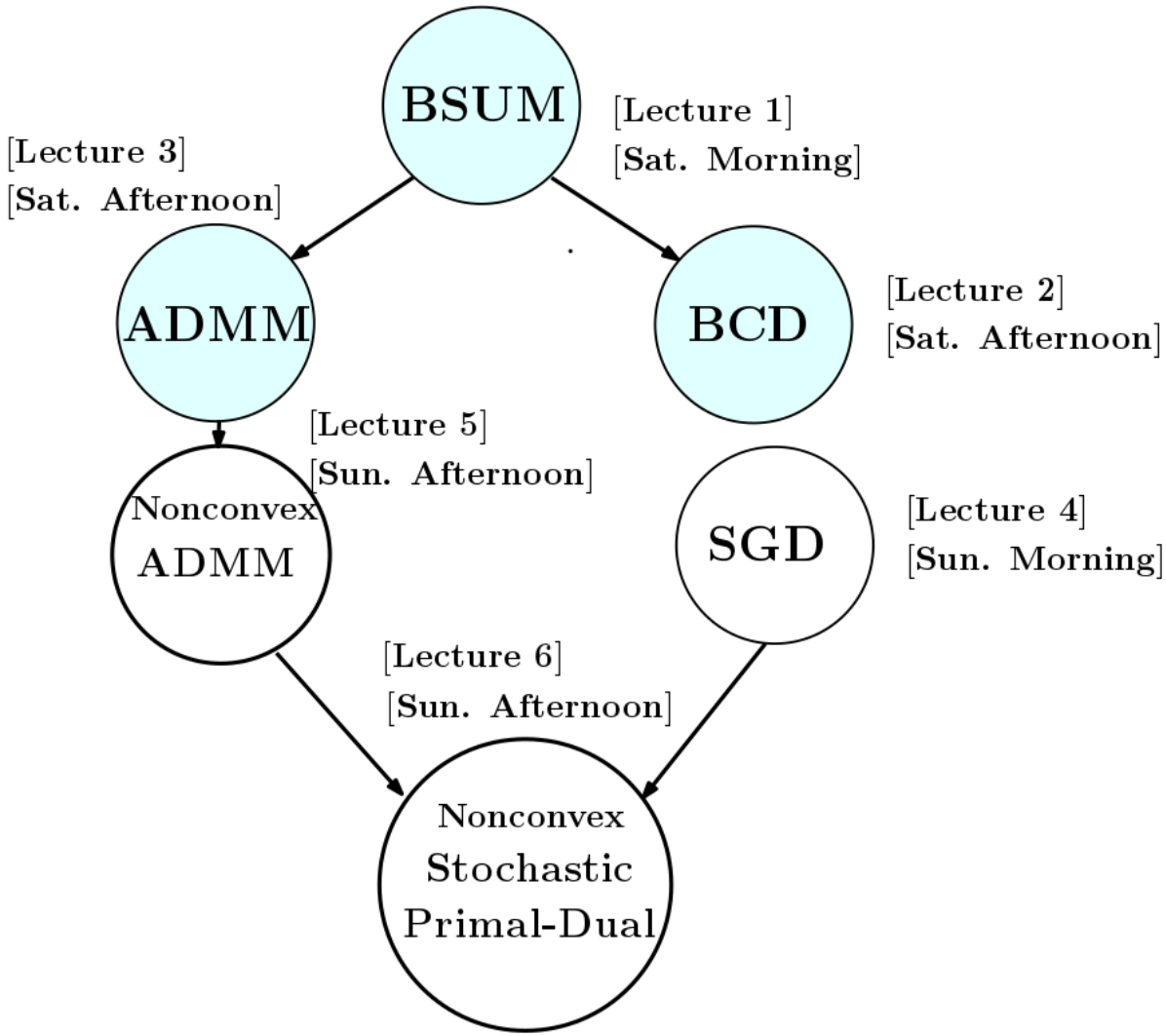
Lecture 3: Optimizing Finite Sums

Mingyi Hong

Iowa State University
IMSE and ECpE Department

June 2016, @ MOA Workshop

Schedule (temporary)



References

1 [Xiao-Zhang 14], "A proximal stochastic gradient method with progressive variance reduction".

2 [Anonymous 15], "Lightweight Stochastic Variance-Reduced ADMM".

3 [Agarwal-Bottou 15], "A Lower Bound for the Optimization for Finite Sum".

4 [Lan 15], "An Optimal Randomized Incremental Gradient Method".

5 [Schwartz-Zhang 13], "Accelerated Mini-Batch Stochastic Dual Coordinate Ascent".

6 [Johnson-Zhang 13], "Accelerated Stochastic Gradient Descent using Predictive Variance Reduction".

7 [Schwartz-Zhang 13], "Stochastic Dual Coordinate Ascent Methods for Regularized Loss Minimization".

8 [Konecny-Qu-Richtarik 14] "Semi-Stochastic Coordinate Descent".

9 [Defazio et al 13] "SAGA: A Fast Incremental Gradient Method With Support for Non-Strongly Convex Composite Objectives".

10 [Le Roux et al] "A Stochastic Gradient Method with an Exponential Convergence Rate for Strongly-Convex Optimization with Finite Training Set".

Agenda

- **What we will do** Present a cluster for recent works on optimizing convex problems of the form:

$$\min \quad \frac{1}{N}\sum_{i=1}^{N} f_i(x), \quad x \in X$$

- **Key point** Decomposition across the objective functions
- **What we hope to achieve**

1. Get a global picture of this line of research
2. Get a comparison of a few key results
3. Identify future research directions

Outline

Outline

The Problem

- The main problem we consider is the following

$$f(x) = \frac{1}{N}\sum_{i=1}^{N} g_i(x) + h(x) \tag{1}$$

- Each g_i represents a data sample; N is large
- In most of the works we will discuss, g_i is smooth and strongly convex
- We are interested in algorithms that
  1. Each time optimize only one component function (cheap iteration)
  2. Can achieve overall linear convergence (fast overall convergence)
- This problem has received significant attention recently, especially among the ML community

Stochastic Gradient Method: Algorithm

- Reduce the dependence on the number of data point
- **The SG method:** $r = 1, \cdots, T$
  1. Randomly pick a data point $i \in \{1, \cdots, N\}$
  2. Compute (sub)gradient $s^r \in \partial f_i(\mathbf{x}^r)$
  3. $\mathbf{x}^{r+1} = \mathbf{x}^r - \alpha^r s^r$
- Very simple and robust
- Starts optimization right away (instead waiting for all (sub)gradients to arrive)

Stochastic Gradient Method: Stepsize and Rate

- The choice of step-sizes: assume $\|s^r\| \leq G$
  1. Fixed stepsize: $\alpha^r = \frac{\|\mathbf{x}^*\|_2}{G\sqrt{T}}$, for all r
  2. Diminishing: $\alpha^r = \frac{\|\mathbf{x}^*\|_2}{G\sqrt{T}}$
  3. Unknown G, $\|\mathbf{x}^*\|$: $\alpha^t = \frac{\beta\|\mathbf{x}^*\|}{G\sqrt{T}}$, for some $\beta > 0$
- The convergence rate:
  1. Fixed stepsize: $\mathbb{E}[f(\bar{x}^r)] - f(\mathbf{x}^*) \leq \frac{G\|\mathbf{w}^*\|_2}{\sqrt{T}}$
  2. Diminishing: $\mathbb{E}[f(\bar{x}^r)] - f(\mathbf{x}^*) \leq \frac{4G\|\mathbf{x}^*\|_2}{\sqrt{T}}$
  3. Unknown G, $\mathbb{E}[f(\bar{x}^r)] - f(\mathbf{x}^*) \leq \frac{4G\|\mathbf{x}^*\|_2}{\sqrt{T}} \max\{\beta, \frac{1}{\beta}\}$, for some $\beta > 0$
- **Main message**: Rate is in the order of $\mathcal{O}(\frac{1}{\sqrt{T}})$; For strongly convex problems the rates will be in the order of $\mathcal{O}(\frac{1}{T})$

The State-of-the-Art

- For strongly convex and smooth problems (modulus μ, L), to achieve ϵ-OPT, the classical optimal first-order methods needs the following number of gradient evaluations

$$\mathcal{O}\left(N\sqrt{L/\mu}\log(1/\epsilon)\right)$$

- A recent work that accelerate the classic SGD (stochastic gradient descent), each iteration a single gradient evaluation, but requires a total of

$$\mathcal{O}\left(\sqrt{L/\mu}\log(1/\epsilon)+\frac{\gamma^2}{\mu\epsilon}\right)$$

gradient evaluations, where $\gamma > 0$ denotes the variance of the stochastic gradient

- **Can we do better**?

Outline

Outline

The SAG algorithm

- Consider a special case of (1) where

$$F(x) = \frac{1}{N}\sum_{i=1}^{N} g_i(x) \tag{2}$$

- The SAG (Stochastic Average Gradient) proposed in [Le Roux 12] is the

Randomly pick an index $i_k \in \{1, \cdots, N\}$

$$y_{i_k}^k = x^k, \quad y_j^k = y_j^{k-1}, \ \forall\ j \neq i_k$$

$$x^{k+1} = x^k - \frac{1}{\eta N}\sum_{i=1}^{N} \nabla g_i(y_i^k)$$

$$= x^k - \frac{1}{\eta N}\sum_{i=1}^{N} \nabla g_i(y_i^{k-1}) + \frac{1}{\eta N}\left(\nabla g_{i_k}(y_{i_k}^{k-1}) - \nabla g_{i_k}(x^k)\right)$$

The SAG algorithm

- A few remarks are ready
  1. y_i^k represents the value of x, in the last time that g_i is sampled
  2. At each iteration only the gradient of one component function is evaluated
- To implement these methods, we need
  1. keep track of the sum $\sum_{i=1}^N \nabla g_i(y_i^{k-1})$;
  2. keep track of the sum $\nabla g_i(y_i^{k-1})$;
- Our main assumption
  1. Each g_i has Lipschtiz gradient with constant L
  2. Each g_i is strongly convex with constant μ
- Let $\sigma^2 = \frac{1}{n}\sum_i \|\nabla g_i(x^*)\|^2$

Convergence Rates

- If $N \geq \frac{8L}{\mu}$, $\eta = 2N\mu$, for all $k \geq N$

$$\mathbb{E}\left[F(x^k) - F^*\right] \leq C\left(1 - \frac{1}{8N}\right)^k$$

$$C = \frac{16L}{3N}\|x^0 - x^*\|^2 + \frac{4\sigma^2}{3N\mu}\left(8\log(1 + \frac{\mu N}{4L} + 1)\right)$$

- Or equivalently, to achieve ϵ-optimality, we need the following number of iterations

$$\mathcal{O}(N\log(1/\epsilon)) = \mathcal{O}\left((N + \frac{L}{\mu})\log(1/\epsilon)\right)$$

Connection with ADMM

- As a side note, SAG is closely related to ADMM
- Consider the global consensus formulation

$$F(x) = \frac{1}{N}\sum_{i=1}^{N} g_i(x_i), \quad \text{s.t. } x_i = x, \ \forall\ i \tag{3}$$

- Let λ_i denote the dual variable for the ith constraint
- We can run an ADMM iteration

$$x^{k+1} = \arg\min \sum_i \frac{\eta}{2}\|x_i^k - x + \lambda_i^k/\eta\|^2$$

$$x_i^{k+1} = \arg\min\ g_i(x_i) + \frac{\eta}{2}\|x_i - x^{k+1} + \lambda_i^k/\eta\|^2, \quad \text{if } i = i_k$$

$$\lambda_i^{k+1} = \eta(x_i^{k+1} - x^{k+1}) + \lambda_i^k, \quad \text{if } i = i_k$$

$$x_j^{k+1} = x_j^k, \quad \lambda_j^{k+1} = \lambda_j^k, \quad \text{if } j \neq i_k$$

- The resulting iteration will be

$$x^{k+1} = \frac{1}{N}\sum_{i=1}^{N}\left(x_i^k - \frac{1}{\eta}\nabla g_i(x_i^k)\right)$$

The SAGA algorithm

- The SAGA algorithm developed in [Defazio 14] improves upon SAG in that
  1. It works for the general form nonsmooth convex problem (1) (not necessarily strongly convex)
  2. It has better convergence rates
- The SAGA algorithm has the following form

Randomly pick an index $i_k \in \{1, \cdots, N\}$

$$y_{i_k}^k = x^k, \quad y_j^k = y_j^{k-1}, \ \forall\ j \neq i_k$$

$$w^{k+1} = x^k - \frac{1}{\eta N}\sum_{i=1}^{N} \nabla g_i(y_i^{k-1}) + \frac{1}{\eta}\left(\nabla g_{i_k}(y_{i_k}^{k-1}) - \nabla g_{i_k}(x^k)\right)$$

$$x^{k+1} = \text{prox}_h^{1/\eta}(w^{k+1}) \qquad \text{(Deal with nonsmooth part)}$$

Comments on SAGA

- The only difference with SAG is that the stepsize of SAGA is N times larger
- Also requires storage of past N gradients
- This is due to a finer analysis, which better explores the uniform randomization used (stepsize inversely proportional to the sampling probabilit)
- Any connection with ADMM?

Connection with ADMM

- Consider the global consensus formulation

$$F(x) = \frac{1}{N}\sum_{i=1}^{N} g_i(x_i), \quad \text{s.t. } x_i = x, \ \forall\ i \tag{4}$$

- Let λ_i denote the dual variable for the ith constraint
- We can run an ADMM iteration

$$x^{k+1} = \arg\min \sum_i \frac{\eta}{2}\|x_i^k - x + \lambda_i^k/\eta\|^2$$

$$x_i^{k+1} = \arg\min\ g_i(x_i) + \frac{\eta}{2N}\|x_i - x^{k+1} + \lambda_i^k/\eta\|^2, \quad \text{if } i = i_k$$

$$\lambda_i^{k+1} = \frac{\eta}{N}(x_i^{k+1} - x^{k+1}) + \lambda_i^k, \quad \text{if } i = i_k$$

$$x_j^{k+1} = x_j^k, \quad \lambda_j^{k+1} = \lambda_j^k, \quad \text{if } j \neq i_k$$

- The resulting iteration will be (where $z_j^k = x_i^k$, and $z_{i_k}^k = x^k$)

$$x^{k+1} = \frac{1}{N}\sum_{i=1}^{N}\left(z_i^k - \frac{1}{\eta}\nabla g_i(x_i^k)\right) + \frac{1}{\eta}\left(\nabla g_{i_k}(x_{i_k}^{k-1}) - \nabla g_{i_k}(x^k)\right)$$

Main Convergence Results

- Suppose $h \equiv 0$ and g_i is strongly convex (modulus μ) and have Lip-gradient (modulus L)
- Choose the following stepsize $\eta = 2(\mu N + L)$
- Then we have the following rates

$$\mathbb{E}[\|x^k - x^*\|^2] \leq \left(1 - \frac{\mu}{2\mu N + L}\right)^k D$$

where

$$D := \|x^0 - x^*\|^2 + \frac{N}{\mu N + L}[f(x^0) - \langle f(x^0) - \langle f'(x^*), x^0 - x^*\rangle - f(x^*)\rangle]$$

- So to achieve ϵ-OPT requires $\mathcal{O}\left((N + L/\mu)\log(1/\epsilon)\right)$ iterations

Main Convergence Results

- Suppose h is present and g_i is not strongly convex
- Choose $\eta = 3L$ (no longer dependent on N)
- Then we have the following rates ($\bar{x}^K$ being the empirical average)

$$\mathbb{E}\left[F(\bar{x}^K)\right] - F(x^*) \leq \frac{4N}{K} D$$

where

$$D = \|x^0 - x^*\|^2 + \frac{2N}{3L}\left[f(x^0) - \langle f'(x^*), x^0 - x^*\rangle - f(x^*)\right]$$

Comments

- Through the lens of ADMM, the results are not too surprising
- Relevant questions to ask
  1. Nonconvex cases? Rates?
  2. Acceleration?
  3. The ADMM-counterpart of SAGA convergent?
  4. Other directions?

Outline

Outline

The Variance Reduction Based Algorithm

- In [Johnson-Zhang 13], the authors propose a now well-known SVRG (stochastic variance reduced gradient) algorithm
- **Main motivation**: Achieve similar convergence rates as SAG, but without requiring additional storage
- Has many followup works in the last two years

The SVRG Algorithm

- The iteration is given by

Iterate: for $s = 1, 2, \cdots$

$\tilde{x} = \tilde{x}_{s-1}$

$\tilde{z} = \frac{1}{N} \sum_{i=1}^{N} \nabla g_i(\tilde{x})$ [averaged gradient, evaluated on a single point]

$x_0 = \tilde{x}$

Iterate: for $t = 1, 2, \cdots$

Randomly pick i_t

$$x_t = x_{t-1} - \frac{1}{\eta} \left(\nabla g_{i_t}(x_{t-1}) - \nabla g_{i_t}(\tilde{x}) + \tilde{z} \right)$$

End

Option I: set $\tilde{x}_s = x_m$

Option II: set $\tilde{x}_s = x_t$ for randomly chosen $t \in \{0, \cdots, m-1\}$

End

Comments on the SVRG Algorithm

- Two time-scale algorithms
- Inner iterations run m-iterations
- Use the same "large" and constant stepsize as the SAGA
- The average of the gradients are evaluated on the same iterate $\tilde{x}$
- The last property removes the need of storing past gradients

Convergence Analysis

- Suppose each g_i has Lip-gradient with modulus L; $h \equiv 0$; f is strongly convex with modulus μ
- Suppose the inner iteration number m is sufficiently large so that

$$\alpha := \frac{\eta^2}{\mu(\eta - 2L)m} + \frac{2L}{\eta - 2L} < 1$$

- Then we have

$$\mathbb{E}[f(\tilde{x}_s)] - f(x^*) \leq \alpha^s [f(\tilde{x}_0) - f(x^*)]$$

- **Remark [Johnson-Zhang 13]** Suppose the condition number $L/\mu = N$; one can take

$$\eta = 10L, \quad m = \mathcal{O}(N)$$

to obtain a convergence rate of $\alpha = 1/2$; Then in total require $N \ln(1/\epsilon)$ number of iterations (compared with standard batch gradient descent $N^2 \ln(1/\epsilon)$)

- In this case, convergence rate is still $\mathcal{O}((N + L/\mu)\log(1/\epsilon))$

A Key Step

- The proof is quite simple
- Key step, bound the "aggregate gradient term"

$$v_t := \nabla g_{i_t}(x_{t-1}) - \nabla g_{i_t}(\tilde{x}) + \tilde{z}$$

- We have the following estimate

$$\mathbb{E}[\|v_t\|^2] \leq 4L[f(x_{t_1}) - f(x^*) + f(\tilde{x}) - f(x^*)]$$

- If $x_t \to x^*$, $\|v_t\| \to 0$

Performance

- Test various algorithms on different problems

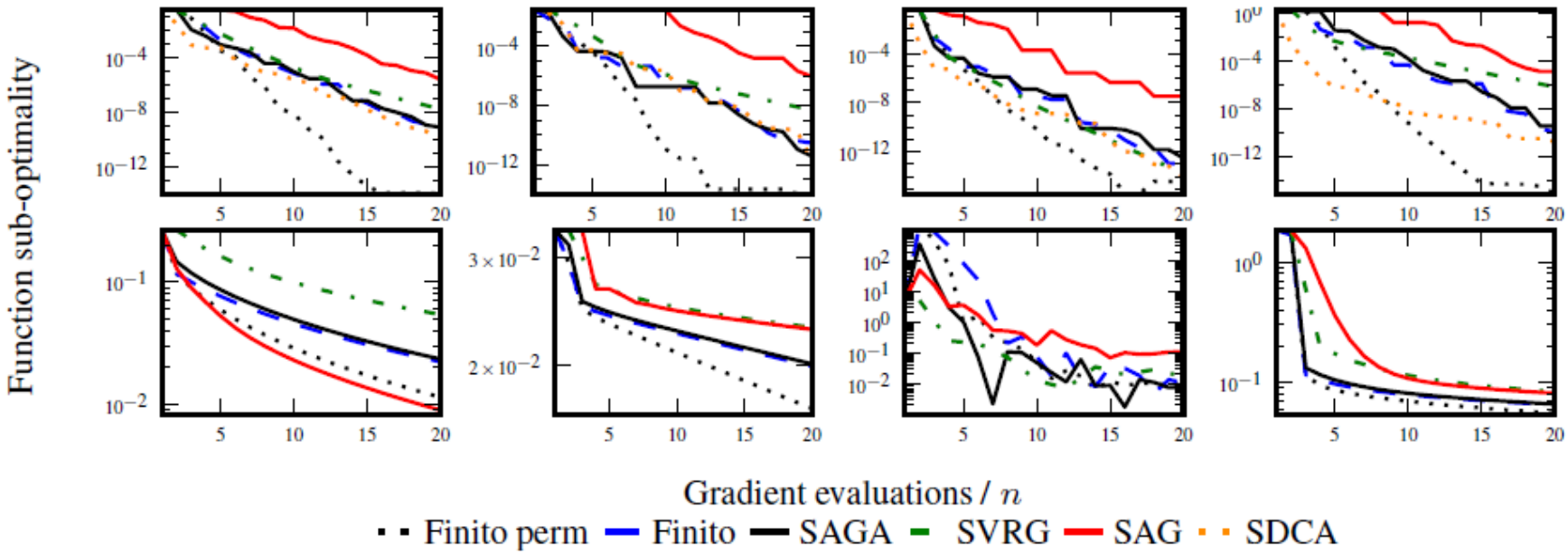


Figure 2: From left to right we have the MNIST, COVTYPE, IJCNN1 and MILLIONSONG datasets. Top row is the L2 regularised case, bottom row the L1 regularised case.

Discussion

- In [Johnson-Zhang 13], the authors mentioned two extensions
  1. Convex case, with $\mathcal{O}(1/\epsilon)$ rate
  2. The algorithm also applies to nonconvex problems
- Comments from [Defazio 14]: "SVRG makes a trade-off between time and space. For the equivalent practical convergence rate it makes 2x-3x more gradient evaluations (than SAGA),.... does not need to store a table of gradients..."
- There have been many extensions of this method
  1. To solve strongly convex problem with nonsmooth term [Xiao-Zhang 14]
  2. To ADMM problem with the first block being a finite sum
  3. To strongly convex problem with coordinate descent [Konecny-Qu-Richtarik 14]
  4. To PCA problems (nonconvex formulation) [Sharmir 15]

Outline

Outline

The SDCA Algorithm

- We briefly discuss the SDCA (stochastic dual coordinate ascent method) [Schwartz-Zhang 13]
- Consider the following problem

$$\min f(x) = \frac{1}{N}\sum_{i=1}^{N} g_i(b_i^T x) + \frac{\lambda}{2}\|x\|^2$$

- Introduce N auxiliary variables

$$\min f(x) = \frac{1}{N}\sum_{i=1}^{N} g_i(c_i) + \frac{\lambda}{2}\|x\|^2, \text{ s.t. } b_i^T x = c_i, \ \forall\ i$$

- Dual problem (where $g_i^*(u) := \max_z(zu - g_i(z))$)

$$\max_{\alpha} D(\alpha) := \left[\frac{1}{N}\sum_{i=1}^{N} -g_i^*(-\alpha_i) - \frac{\lambda}{2}\left\|\frac{1}{\lambda N}\sum_{i=1}^{N}\alpha_i b_i\right\|^2\right]$$

The SDCA Algorithm

- It is known that at optimal $x^* = \frac{1}{\lambda N}\sum_{i=1}^N \alpha_i^* b_i$
- The SDCA algorithm basically performs stochastic CD on $D(\alpha)$
- The detailed algorithm is given below (A good choice of $T_0 = T/2$)

Iterate: for $t = 1, 2, \cdots, T$

Randomly pick i

Find $\nabla\alpha_i$ to maximize

$$-g_i^*(-(\alpha_i^{r-1} + \nabla\alpha_i)) - \frac{\lambda N}{2}\|x^{t-1} + (\lambda N)^{-1}\nabla\alpha_i b_i\|^2$$

Let $\alpha^t = \alpha^{t-1} + \nabla\alpha_i e_i$

Let $x^t = x^{t-1} + (\lambda N)^{-1}\nabla\alpha_i x_i$

End

Output :

Let $\bar{\alpha} = \frac{1}{T-T_0}\sum_{i=T_0+1}^T \alpha^{t-1}$

Let $\bar{w} = \frac{1}{T-T_0}\sum_{i=T_0+1}^T w^{t-1}$

The Convergence Analysis

- If g_i is Q-Lip for all i; To obtain $\mathbb{E}[f(\bar{x}) - f(\bar{\alpha})] \leq \epsilon$, we need

$$T \geq T_0 + N + \frac{4Q^2}{\lambda\epsilon}$$

- If g_i has L-Lip gradient; To obtain $\mathbb{E}[f(\bar{x}) - f(\bar{\alpha})] \leq \epsilon$, we need

$$T \geq \left(N + \frac{L}{\lambda}\right) \log\left(N + \frac{L}{\lambda}\frac{1}{\epsilon}\right)$$

Outline

Outline

Introduction

- **Key Question** What is the performance lower bound for solving problem

$$\min f(x) := \quad \frac{1}{N}\sum_{i=1}^{N} g_i(x), \quad x \in X$$

- All the results so far has the similar complexity estimate (for strongly convex problems)

$$\mathcal{O}((N + L/\mu)\log(1/\epsilon)) := \mathcal{O}((N + \kappa)\log(1/\epsilon))$$

- Can we further improve the rates?

A Performance Lower Bound

- Recently, [Agarwal-Bottou 15] attempts to provide a characterization
- **Main Results**
  1. If $X \in \mathbb{R}^N$ and f_i is strongly convex for each i
  2. Each step of the algorithm follows certain "Incremental First-Order Oracle"
  3. $\exists\ f$ such that all algorithms (following the oracle) requires at least

$$\mathcal{O}\left(N + \sqrt{N(\kappa-1)}\log(1/\epsilon)\right)$$

  calls to the IFO to achieve an ϵ-OPT

- Comparing to the optimal gradient method (calls to the IFO)

$$\mathcal{O}\left(N\sqrt{\kappa}\log(1/\epsilon)\right)$$

Setup

- Let us define the class of function $\mathcal{F}_N^{\mu,L}$ as the class of all convex function f in (1) where each g_i is $L-\mu$ smooth and convex
- **The Incremental First-order-Oracle (IFO)**: For a function $f \in \mathcal{F}_N^{\mu,L}$, the IFO takes as input a point x and index $i \in \{1, 2, \cdots, N\}$, and returns the pair $(g_i(x), g_i'(x))$
- **The IFO Algorithm**: An optimization algorithm is an IFO algorithm if its specification does not depend on the cost function f other than through calls to an IFO
- **Question**: For any IFO algorithm, how many times (T) the IFO has to be invoked so that and ϵ-solution is achieved, i.e.,

$$\|x^T - x^*\| \leq \epsilon \|x^*\|$$

for all $f \in \mathcal{F}_N^{\mu,L}$

Main Result

- We have the following main result in [Agarwal-Bottou 15]
- Consider an IFO algorithm for problem (1) that performs T calls to the IFO. Then for any $\gamma > 0$, $\exists\ f \in \mathcal{F}_N^{\mu,L}$ s.t. $\|x_f^*\| = \gamma$, and

$$\|x_f^* - x^T\| \geq \gamma q^{2\ell}, \quad \text{with } q = \frac{\sqrt{1 + \frac{\kappa-1}{N}} - 1}{\sqrt{1 + \frac{\kappa-1}{N}} + 1}$$

  and $\ell = 0$ if $T < N$, $\ell = T/N$ otherwise
- A direct consequence: Consider an IFO that guarantees $\|x_f^* - x^T\| \leq \epsilon \|x_f^*\|$ for any $\epsilon < 1$. Then $\exists\ f \in \mathcal{F}_N^{\mu,L}$ on which the algorithm must perform at least the following # of iterations

$$T = \mathcal{O}\left(N + \sqrt{N(\kappa - 1)}\log(1/\epsilon)\right)$$

Remarks

- We need to be careful about the family of problems considered here
- **Statement 1**. The SVRG, SAG, SAGA are included in the IFO algorithm [Agarwal-Bottou 15]
- **Statement 2**. The SDCA are not included in the IFO algorithm, as they require access to the gradients of the conjugate of g_i [Agarwal-Bottou 15]
- However, a gap: we need to be careful about how the indices are selected.
  1. In the definition of IFO algorithm the index selection is arbitrary
  2. In the proof the authors used deterministic cyclic index selection
  3. Does this imply the same bound for the random index selection?

Outline of the Proof

- The proof mainly follows [Nemirovsky-Yudin 1983]
- The family of problems used to establish the lower bound is

$$f(x) = \frac{\mu}{2}\|x\|^2 + \frac{1}{N}\sum_{i=1}^{N} h_i(Q_i^T x)$$

where

1. $h_i(x) \in \mathcal{S}^{0,L-\mu}$ (0-strongly convex, $L-\mu$ smooth functions)
2. Q_i's belong to a family of orthogonal matrices

$$Q_i = [e_i, e_{N+i}, e_{2N+i}, \cdots,]$$

Outline of the Proof

- One can show

$$f(x) = \frac{1}{N}\sum_{i=1}^{N}\left[\frac{N\mu}{2}\|Q_i^T x\|^2 + h_i(Q_i^T x)\right] := \frac{1}{N}\sum_{i=1}^{N} g_i(Q_i^T x)$$

- Each $g_i \in \mathcal{S}^{N\mu, L-\mu+N\mu}$
- Now notice that due to the definition of Q_i, $g_i(Q_i^T x)$ is independent with $g_j(Q_j^T x)$ for all $i \neq j$ (i.e., the minimization of $f(x)$ is a separable problem)
- Therefore minimizing $f(x)$ is equivalent to find

$$x^* = \sum_{i=1}^{N} Q_i x_i^*, \quad x_i^* = \arg\min_x g_i(Q_i^T x)$$

Outline of the proof

- View the algorithm as a complicated setup whose purpose is to optimize g_i
- Utilize the classical result by Nemirovsky-Yudin
- Consider a first order black box optimization algorithm that performs $T > 0$ calls and returns x^T; For any $\gamma > 0$, $\exists\ g \in \mathcal{S}^{\mu,L}$ such that $\|x_f^*\| = \gamma$ and

$$\|x_g^* - x^T\| \geq \gamma q^{2T}, \quad q = \frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \tag{5}$$

- In this proof, each component function will be chosen independently to witness the previous lower bound (requires some further details, omitted in this presentation)

Comments

- Maybe worth studying this line of work
- The stochastic version of the problem is not solved (to my understanding)
- Deterministic algorithms matching the above bounds?
- How about lower bounds (and algorithms) for the convex cases?

Conclusion

- We have surveyed a series of works on optimizing a problem with finite sum
- This line of research has received significant attention recently among the ML community
- It is useful to put them together and compare their similarities in terms of algorithmic steps, convergence conditions, convergence proof as well as convergence rates
- Key research questions in this line of work are
  1. Design fast algorithms (with theoretical guarantees)
  2. Extending existing algorithms to wider class of problems
  3. Deriving performance lower bounds

Future Works

- Lower bounds
  1. Bridging the gap between deterministic and random methods
  2. Lower bounds for general convex problems
- Bound-achieving algorithms (both deterministic and randomized); still gaps here, even for strongly convex problems (some recent results by Lan et al addresses this issue)
- Nonconvex problems with finite sum?
- Exploring the relationship among different methods?
- Anything else?

Thank You!